

EVENTSYS: A DYNAMIC CAMPUS EVENT MANAGEMENT AND SMART TICKETING PLATFORM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

REDDIPALLI JAHEER (VTU24438)

10211CS224

Full Stack Application Development

Slot : E1-B2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ""EventSys: A Dynamic Campus Event Management and Smart Ticketing Platform"" by REDDIPALLI.JAHEER (VTU24438) has been carried out in the Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Coordinator

Dr.Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(REDDIPALLI.JAHEER)

Date:06/05/2026

ABSTRACT

This project focuses on the design and development of a full-stack web application that integrates both front-end and back-end technologies to deliver a seamless user experience. The primary objective of the system is to provide an efficient, scalable, and user-friendly platform for managing and interacting with data in real time. The application is developed using modern web technologies, where the front end ensures an interactive and responsive user interface, while the back end handles business logic, data processing, and secure communication with the database. The system follows a modular architecture, enabling easy maintenance, scalability, and future enhancements. Key features of the application include user authentication, dynamic data handling, and responsive design, ensuring accessibility across multiple devices. The integration of APIs facilitates smooth communication between the client and server, improving overall system performance.

Keywords:

- Full Stack Development
- Web Application

- Front-End Development
- Back-End Development
- Database Management
- RESTful APIs
- User Authentication
- Responsive Design
- Client-Server Architecture

LIST OF FIGURES

1.1	System Overview Diagram	3
3.1	Backend Architecture	8
5.1	User login page	16
5.2	Upcomming Events	17
5.3	Ticket booking page	18
5.4	Payment Details	19

LIST OF TABLES

5.1	Comparison of Event Management Platforms	15
6.1	Mapping of CEP Attributes	23
6.2	Mapping of CEA Attributes	24
6.3	SDG Mapping	24

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create,Read, Update, Delete
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.2 Backend Implementation	7

3.3	Database Implementation	8
4	METHODOLOGIES	10
4.1	Project Architecture	10
4.2	DataFlow Diagram	10
4.3	Module 1: User Authentication Registration	11
4.4	Module 2:Event Management (Admin Dashboard) . .	11
4.5	Module 3:Booking Notification System	12
5	RESULTS AND DISCUSSIONS	14
5.1	Screenshots of the system and Results	15
5.2	User Login Page	15
5.3	Upcomming Events	16
5.4	Ticket booking Page	17
5.5	Payment	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	20
6.1	Conclusion	20
6.2	Future Enhancements	21
6.3	Existing Applications and Review	21
	References	23
	Source Code	26

6.4	Source Code	26
-----	-----------------------	----

Chapter 1

INTRODUCTION

1.1 Introduction

The application follows a client-server architecture, where the client side interacts with the server through APIs. These APIs ensure smooth data exchange and enable real-time updates. The use of modern technologies and frameworks enhances the scalability, flexibility, and maintainability of the system

1.2 Aim of the Project

The aim of this project is to design and develop a full-stack web application that provides an efficient, scalable, and user-friendly platform for managing and interacting with data. The project focuses on integrating front-end and back-end technologies to ensure seamless communication, real-time data processing, and a responsive user interface.

1.3 Scope of the Project

The scope of this project covers the design, development, and implementation of a full-stack web application that enables efficient data management and user interaction through a responsive and interactive interface. The system supports core functionalities such as user registration, login authentication, data storage, and real-time data retrieval using a structured database.

The project includes both front-end and back-end development, ensuring seamless communication through RESTful APIs within a client-server architecture. It focuses on providing a secure and scalable solution that can handle multiple users simultaneously while maintaining performance and reliability.

Framework Used

The project is developed using modern full-stack development frameworks and technologies to ensure efficiency, scalability, and maintainability.

- **Front-End Framework:** Used for designing interactive and responsive user interfaces that enhance user experience.
- **Back-End Framework:** Handles server-side logic, request pro-

cessing, and communication with the database.

- **Database Framework:** Used for storing, managing, and retrieving application data efficiently.
- **API Framework:** Enables smooth communication between the client and server through RESTful services.
- **Authentication Framework:** Provides secure user login and access control mechanisms.

These frameworks work together to create a robust and scalable web application, ensuring smooth performance, reliability, and flexibility for future enhancements.

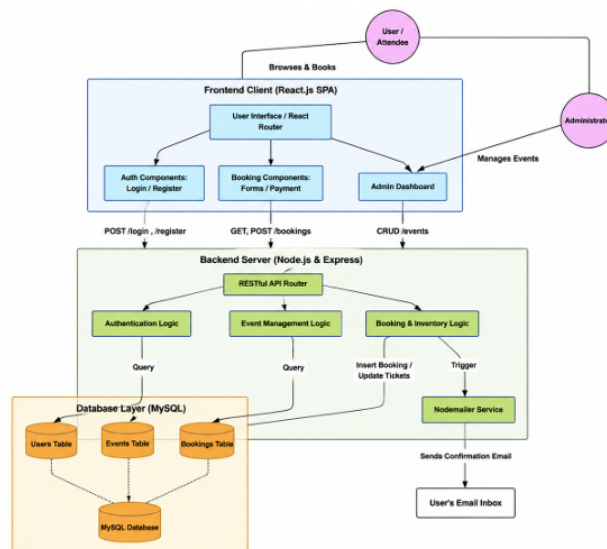


Figure 1.1: System Overview Diagram

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The market for event booking platforms is highly competitive, featuring industry leaders such as Eventbrite, Meetup, and BookMyShow

- **Eventbrite:** Offers extensive tools for organizers but can be highly complex and introduces significant transaction fees, making it less ideal for localized or institutional events.
- **Meetup:** Focuses primarily on community-building and recurring group meetups. It lacks the streamlined ticketing and pricing structure needed for standalone commercial or departmental events.
- **BookMYShow:** A dominant player in the entertainment ticketing sector, but its architecture is heavily tailored for large-scale cinema and concert events, making it overkill for smaller custom event management. The proposed Event Booking Application differentiates itself by offering a lightweight, highly customizable, and

cost-effective solution specifically tailored for institutional, departmental, or mid-sized community events without the overhead of enterprise-level platforms. Additionally, the proposed system provides a simplified user interface and streamlined workflow that ensures ease of use for both administrators and users. Unlike existing platforms that may overwhelm users with complex features, this application focuses on essential functionalities such as event creation, booking, and confirmation, making it more accessible for academic and small-scale organizational use. Furthermore, the application is designed with flexibility and scalability in mind, allowing future enhancements such as secure authentication, real-time payment integration, and advanced filtering options. This makes the system adaptable to evolving user needs while maintaining a cost-effective and efficient solution compared to large commercial platforms.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend environment is initialized using Create React App (CRA). Key dependencies include 'react-router-dom' for client-side routing, 'axios' for HTTP API requests, 'framer-motion' for UI animations, and 'react-toastify' for user feedback notifications. The development environment requires Node.js and npm to be installed locally

3.1.2 Structure of the Project The frontend source code (located in the 'src' directory) is highly modularized:

- **App.js:** The root component handling global state (user/admin login status) and application routing
- **Components:** Contains reusable UI components such as Navbar, Header, BookingForm, Login, UserLogin, UserRegister, Payment, and Confirmation.

- **Index.js** The entry point that renders the React application into the browser DOM.

3.1.3 Execution Procedure To execute the frontend: 1. Navigate to the project root directory in the terminal. 2. Execute `npm install` to install all necessary dependencies. 3. Execute `npm start` to launch the development server. The application will be accessible at `http://localhost:3000`.

3.2 Backend Implementation

3.2.1 Environment Setup The backend is built using Node.js. The environment is initialized with `npm init` and relies on several critical packages: `express` for setting up the web server and routing, `mysql2` for database connectivity, `cors` for Cross-Origin Resource Sharing, `dotenv` for managing sensitive environment variables, and `nodemailer` for automated email services.

3.2.2 Structure of the Project The backend architecture is centralized in the `backend/server.js` file, which includes:

- Server configuration and middleware setup(`cors`, `express.json`).
- MySQL database connection initialization and automatic table creation

- RESTful API routes categorized into Users(/register, /login), Events (/events), and Bookings (/bookings).

3.2.3 Execution Procedure To execute the backend: 1. Navigate to the backend directory. 2. Execute npm install to install backend dependencies. 3. Ensure a MySQL server is running locally with the credentials specified in the configuration. 4. Execute node server.js to start the backend server, which will run on http://localhost:5001

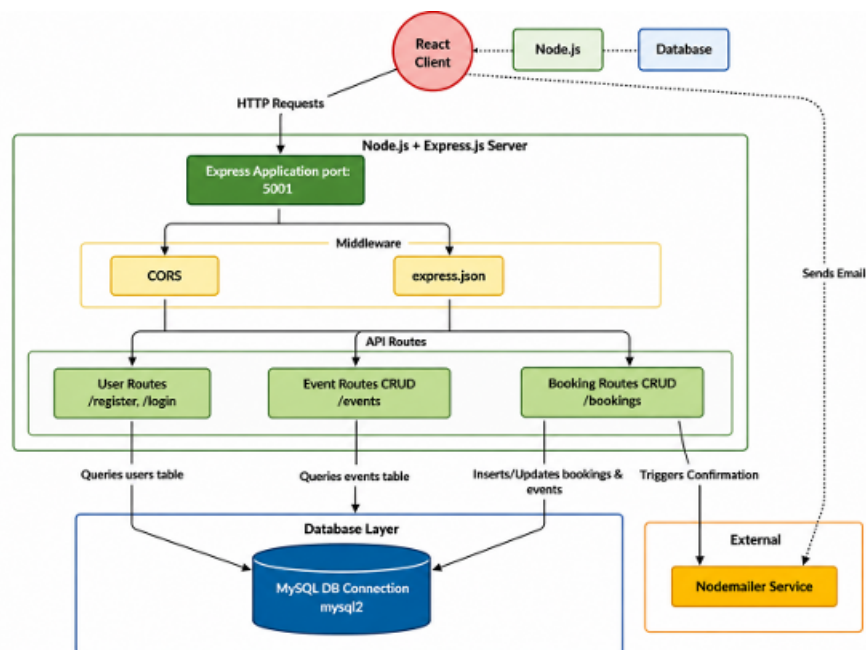


Figure 3.1: Backend Architecture

3.3 Database Implementation

3.3.1 Database Configuration The application utilizes MySQL for persistent data storage. The database is named eventdb. The connection is established securely using the mysql2 library, connecting to lo-

calhost on the default MySQLport utilizing root credentials

3.1.2 Schema Structure The schema is relationally designed to ensure data integrity and prevent redundancy across user profiles, available events, and transactional booking records.

3.1.3 Tables created

- **USER:**Stores user credentials. Columns include id (Primary Key, Auto Increment), name, email (Unique), and password.
- **Events:**Stores event details. Columns include id (Primary Key), name, department, date, venue, price, and tickets (representing available capacity).
- **Bookings:** Stores transaction records. Columns include id (Primary Key), name, email, department, tickets (quantity booked), and event id (linking to the events table).

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a standard Client-Server architecture. The React frontend acts as the client, rendering the user interface as a Single Page Application (SPA). It communicates asynchronously via Axios HTTP requests to the Node.js/Express backend. The backend serves as the intermediary business logic layer, processing requests, interacting with the MySQL database via SQL queries, and returning JSON responses back to the client.

4.2 DataFlow Diagram

1. The User interacts with the React UI to submit a booking form
2. The React component dispatches an Axios POST request to the backend API (/bookings).

3. The Express server receives the request, validates the data payload, and executes a MySQL INSERT query to permanently record the booking.
4. The server executes an UPDATE query to accurately decrement the available ticket count in the events table.
5. The server utilizes Nodemailer to dispatch an automated confirmation email to the user's provided email address.
6. A success response is returned to the client, triggering a UI state update to display a confirmation message via React Toastify.

4.3 Module 1: User Authentication Registration

This module handles the creation of new user accounts and the authentication of existing users. It features form validation on the frontend and secure credential verification against the MySQL users table on the backend, accurately managing login states within the React application to protect private routes.

4.4 Module 2: Event Management (Admin Dashboard)

This module provides administrators with a protected interface to perform comprehensive CRUD operations on the event catalog. Ad

ministrators can create new events by specifying details like name, date, venue, price, and ticket capacity, as well as update existing event details or remove canceled events.

4.5 Module 3:Booking Notification System

This module enables authenticated users to select events and reserve tickets. It ensures transactional integrity by recording the booking, updating the available ticket inventory in real-time within the database, and automatically triggering a booking confirmation email using an SMTPservice (Nodemailer).

Advantages of this Project

- **Scalability:** The decoupled React frontend and Express backend can be scaled and maintained independently
- **Real-Time Inventory Control:** Ticket availability is updated synchronously upon booking, effectively preventing double-booking or over-booking
- **Automated Communication:**Integrated email notifications enhance the user experience by providing immediate, reliable booking confirmations

Disadvantages

- **Security Constraints:** Passwords are currently managed in plain text; implementing bcrypt hashing is required for production-level security.
- **Payment Integration:** The payment process is visually simulated and currently lacks integration with a live third-party payment gateway (e.g., Stripe, Razorpay) for actual financial processing.

This project also promotes modular development by clearly separating frontend, backend, and database layers, which improves maintainability and future scalability. The user-friendly interface ensures smooth navigation, making it easy for users to browse events and complete bookings efficiently. Additionally, the system supports real-time interaction between components, enhancing responsiveness and overall performance. Another limitation of the system is that it is currently deployed only in a local environment, which restricts accessibility to external users. The system also lacks advanced features such as role-based access control and detailed analytics for administrators. Furthermore, error handling and validation mechanisms can be enhanced to improve system robustness and reliability in real-world scenarios.

Chapter 5

RESULTS AND DISCUSSIONS

TheEventBookingApplicationsuccessfully fulfills its primary functional requirements. Users can intuitively navigate the platform, register accounts securely, and book event tickets without friction. Administrators have full, reliable control over event listings through the dedicated dashboard. The integration of React Router provides a seamless Single Page Application experience, resulting in fast navigation without full page reloads. The Express backend efficiently processes concurrent API requests, and the MySQL database accurately maintains data consistency, particularly during ticket quantity deductions. During local testing, the Nodemailer integration reliably delivered email confirmations immediately upon successful bookings.

article

[margin=1in]geometry array longtable

Feature	Eventbrite	BookMyShow	Proposed App
Custom Branding	Limited	Limited	High
Transaction Fees	High	Moderate	None (Self-Hosted)
Setup Complexity	High	Moderate	Low
Target Audience	Commercial Events	Entertainment Sector	Institutional / Departmental
Data Ownership	Third-Party	Third-Party	Fully Owned
Flexibility	Limited Customization	Limited Customization	Highly Customizable
Cost Efficiency	Low	Moderate	High

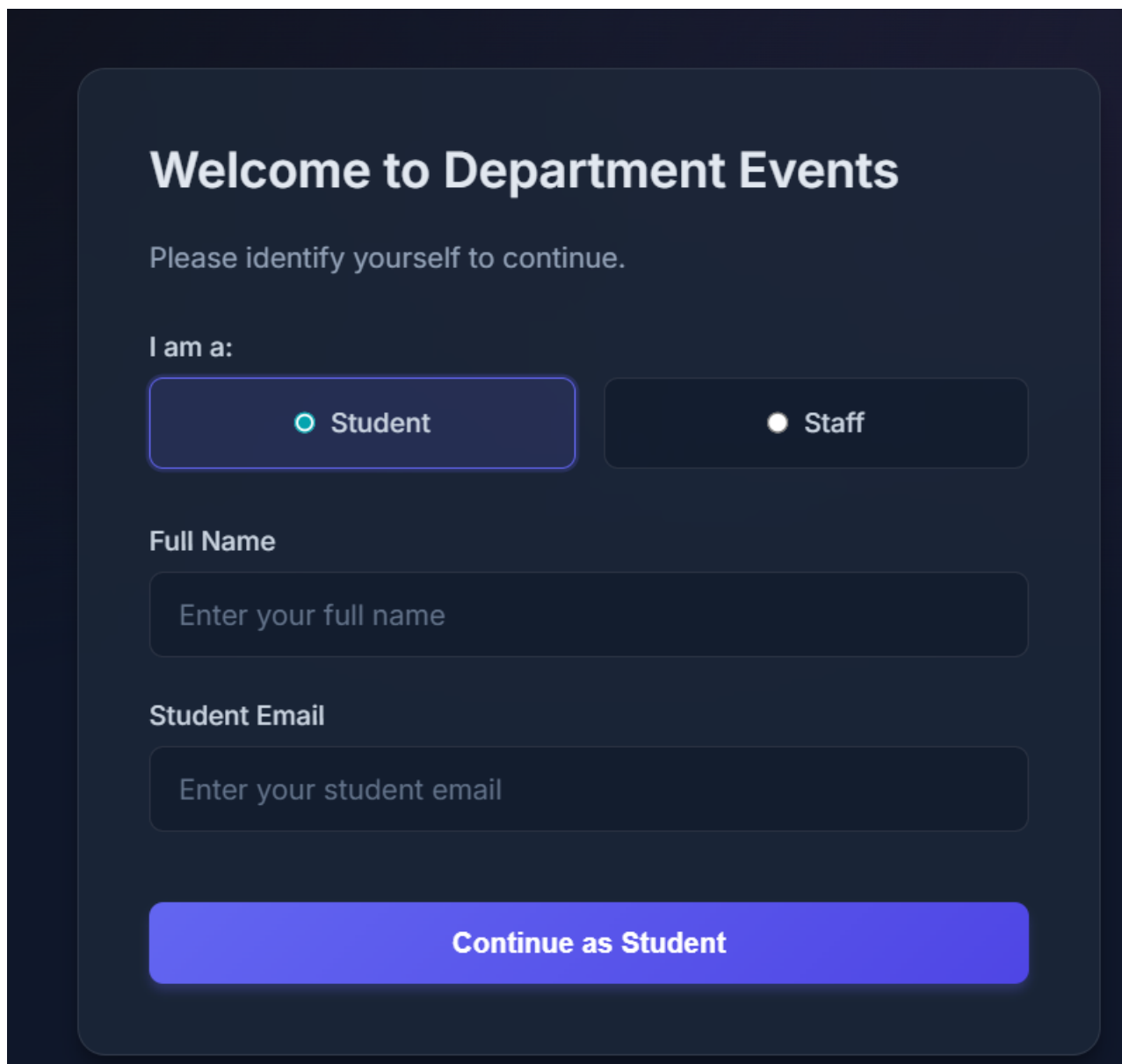
Table 5.1: Comparison of Event Management Platforms

5.1 Screenshots of the system and Results

The developed Event Booking Application is currently deployed in a local environment (localhost) and includes the following modules:

5.2 User Login Page

This page allows registered users to securely log into the system using their credentials. It ensures proper authentication before granting access to booking features.



The image shows a user login page with a dark blue background. At the top, the heading "Welcome to Department Events" is displayed in white. Below it, a prompt "Please identify yourself to continue." is shown. The "I am a:" section contains two buttons: "Student" (selected with a green dot) and "Staff" (unselected with a white dot). Below these are input fields for "Full Name" and "Student Email", both with placeholder text "Enter your full name" and "Enter your student email" respectively. At the bottom, a large blue button labeled "Continue as Student" is visible.

Figure 5.1: User login page

5.3 Upcomming Events

College events play a vital role in shaping a student's overall development beyond academics. They provide opportunities for students to showcase their talents, enhance leadership skills, and build confidence through activities such as cultural festivals, technical symposiums, workshops, and sports competitions.

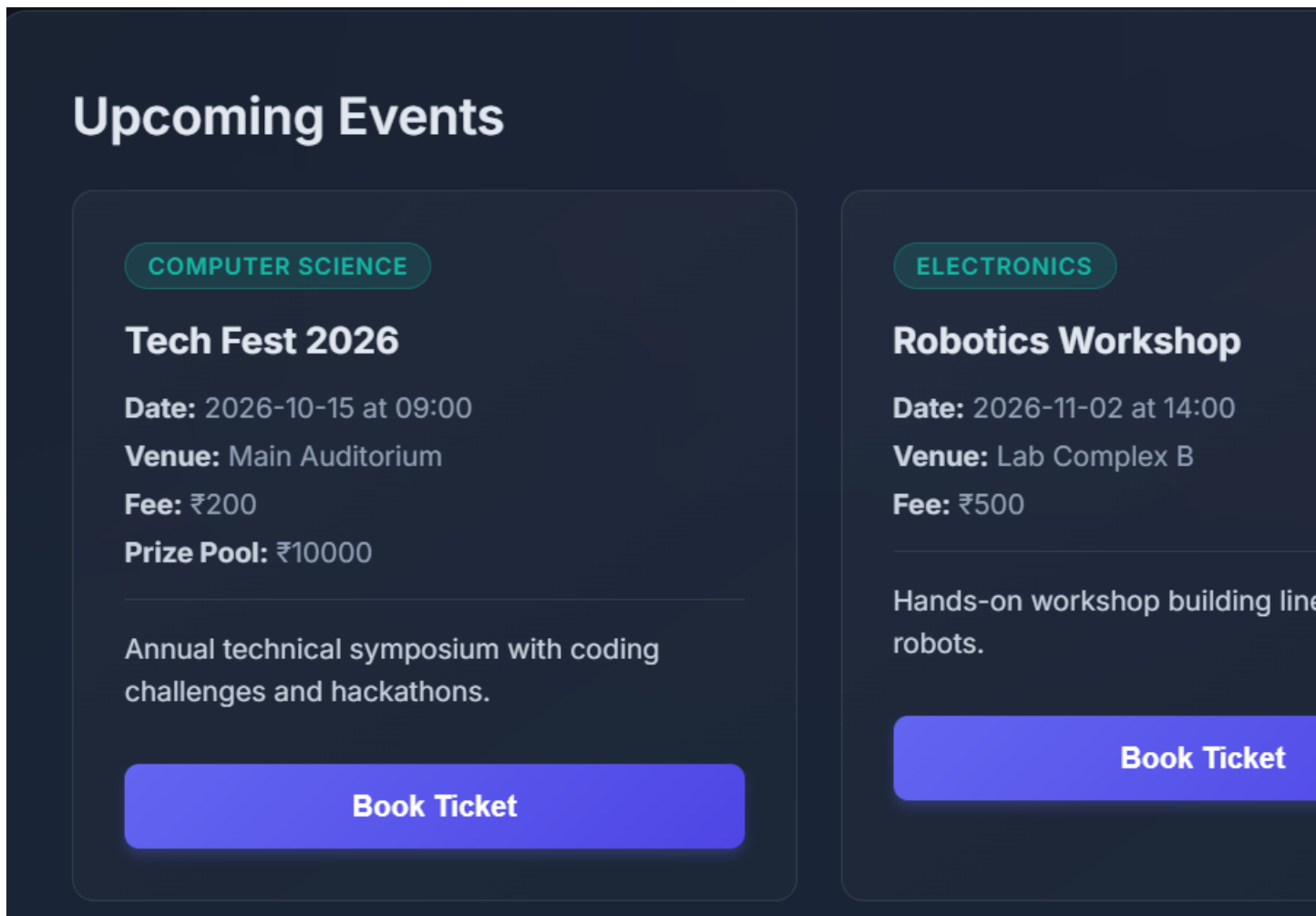


Figure 5.2: Upcomming Events

5.4 Ticket booking Page

This page allows users to view available events and select tickets for booking. It provides real-time updates on ticket availability and event details.

5.5 Payment

The payment page collects user payment details to confirm ticket booking.

Book Ticket

Event: Robotics Workshop

Name

jaheer

Student Email Address

vtu24438@veltech.edu.in

Registration Fee: ₹500

* Payment to be collected at the venue.

Cancel

Confirm Booking

Figure 5.3: Ticket booking page



Booking Confirmed!

Your ticket has been successfully booked.

Booking Summary

Name:	jaheer
Email:	vtu24438@veltech.edu.in
Event:	Robotics Workshop
Department:	Electronics
Date & Time:	2026-11-02 @ 14:00
Venue:	Lab Complex B
Amount Due:	₹500

[Back to Dashboard](#)

Figure 5.4: Payment Details

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Event Booking Application demonstrates the effectiveness of utilizing modern web technologies to build a robust and interactive platform. By integrating React.js, Node.js, Express, and MySQL, the system provides a seamless experience for users, from browsing events to booking tickets and receiving confirmation emails. The application successfully implements key functionalities such as user authentication, event management, ticket booking, payment simulation, and email notifications. The system is efficient, user-friendly, and suitable for institutional event management.

6.2 Future Enhancements

- **Secure Authentication:** Implement JSON Web Tokens (JWT) and bcrypt to enhance security and ensure safe login mechanisms
- **Payment Gateway Integration:** Integrate real-time payment services such as Stripe or PayPal for secure transactions
- **Advanced Filtering:** Enable users to search, sort, and filter events based on categories, date, and keywords
- **Digital Ticketing:** Generate QR codes for each booking and include them in confirmation emails for easy verification
- **Cloud Deployment:** Deploy the system using cloud platforms such as AWS, Vercel, or Netlify.

6.3 Existing Applications and Review

- **Eventbrite** <https://www.eventbrite.com> Eventbrite is a widely used event management platform that provides ticketing, analytics, and promotion tools. However, it involves service charges and may be complex for beginners.
- **BookMyShow** <https://in.bookmyshow.com> BookMyShow is popular in India for booking movie and event tickets. It offers

a smooth user interface but is mainly focused on entertainment events

- **Meetup** <https://www.meetup.com> Meetup enables users to join communities and attend events. It is useful for networking but lacks strong ticketing and payment integration.

[12pt]report [margin=1in]geometry array setspace

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with booking constraints and validation
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

Table 6.1: Mapping of CEP Attributes

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 6.2: Mapping of CEA Attributes

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and learning platforms
SDG 9	Innovation	Promotes use of modern React-based systems
SDG 10	Reduced Inequalities	Accessible system for all types of users
SDG 11	Sustainable Communities	Encourages organized and efficient event management

Table 6.3: SDG Mapping

References

1. React Official Documentation: <https://react.dev/learn>
2. React Router
Documentation:<https://reactrouter.com/en/main>
3. Node.js Official
Documentation:<https://nodejs.org/en/docs>
4. Express.js Framework:<https://expressjs.com/>
5. MySQL Official
Documentation:<https://dev.mysql.com/doc/>
6. MySQL2 Node.js
Driver:<https://www.npmjs.com/package/mysql2>
7. Nodemailer Documentation:<https://nodemailer.com/>
8. Axios HTTP
Client:<https://axios-http.com/docs/intro>

Source Code

6.4 Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internal Department Event Ticket Booking</title>
  <link rel="stylesheet" href="styles.css">

  <!-- React & ReactDOM -->
  <script crossorigin
    src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script crossorigin
    src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

  <!-- Babel -->
  <script
    src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>

<body>
  <div id="root"></div>

  <!-- Components -->
  <script type="text/babel" src="components/Login.js"></script>
  <script type="text/babel" src="components/CreateEvent.js"></script>
  <script type="text/babel" src="components/EventList.js"></script>
  <script type="text/babel" src="components/BookingForm.js"></script>
```

```
<script type="text/babel" src="components/Confirmation.js"></script>

<!-- Main App -->
<script type="text/babel" src="App.js"></script>
</body>
</html>
```